

❑ Milestone 1 —Modélisation orientée objet

Sprint V1 -Al Qalam Stock Manager Thème : class `__init__` ·
encapsulation ·JSON ·Tkinter

Objectif

Construire le **squelette complet** de l'application : les classes métier, la persistance JSON et l'interface graphique de base permettant d'afficher, ajouter, et gérer les entrées/sorties de stock.

Ce que vous allez construire

```
alqalamv1/
├── main.py                ← point d'entrée : py -3 main.py
├── config.py              ← constantes globales (couleurs, chemins, version)
├── models/
│   ├── produit.py        ← classe Produit
│   ├── categorie.py      ← classe Categorie
│   └── mouvement.py      ← classe Mouvement (entrée/sortie)
├── services/
│   └── stock_service.py  ← StockService (logique métier + JSON)
├── ui/
│   ├── app.py            ← AlQalamApp(CTk) — fenêtre principale
│   └── frames/
│       ├── stock_frame.py ← tableau Treeview + boutons
│       └── dialogs.py    ← dialogues modaux
└── data/
    └── stock.json        ← créé automatiquement au 1er lancement
```

Concepts POO mis en œuvre

Concept	Où	Ce que ça fait
class + <code>__init__</code>	Produit, Mouvement	Définit la structure d'un objet
Encapsulation (<code>_</code>)	StockService	Données internes non modifiables directement
Méthodes d'instance	<code>produit.est_complet()</code>	Comportement attaché à l'objet
Méthodes de classe	<code>Produit.from_dict()</code>	Constructeur alternatif

Concept	Où	Ce que ça fait
Séparation des couches	models/ · services/ ·ui/	Chaque couche a une seule responsabilité

Guide de réalisation étape par étape

Étape 1 — Les modèles (20 min)

La classe `Produit` représente un article du stock. Elle est au cœur de toute l'application.

models/produit.py

class `Produit`:

"""Un article commercialisé par Al Qalam."""

```
def __init__(self, ref: str, nom: str, categorie: str,
              prix_achat: float, prix_vente: float,
              qte: int, seuil_min: int = 5):
    self.ref = ref
    self.nom = nom
    self.categorie = categorie
    self.prix_achat = prix_achat
    self.prix_vente = prix_vente
    self.qte = qte
    self.seuil_min = seuil_min
```

— Méthodes métier

```
def est_en_alerte(self) -> bool:
    """True si la quantité est sous le seuil minimum."""
    return self.qte < self.seuil_min
```

```
def valeur_stock(self) -> float:
    """Valeur d'inventaire : quantité × prix d'achat."""
    return self.qte * self.prix_achat
```

```
def marge_unitaire(self) -> float:
    """Marge brute par unité vendue."""
    return self.prix_vente - self.prix_achat
```

— Sérialisation JSON

```
def to_dict(self) -> dict:
    """Convertit le produit en dictionnaire pour la sauvegarde JSON."""
```

```

    return {
        "ref": self.ref, "nom": self.nom, "categorie": self.categorie,
        "prix_achat": self.prix_achat, "prix_vente": self.prix_vente,
        "qte": self.qte, "seuil_min": self.seuil_min,
    }

    @classmethod
    def from_dict(cls, d: dict) -> "Produit":
        """Crée un Produit depuis un dictionnaire (chargement JSON)."""
        return cls(
            ref=d["ref"], nom=d["nom"], categorie=d["categorie"],
            prix_achat=d["prix_achat"], prix_vente=d["prix_vente"],
            qte=d["qte"], seuil_min=d.get("seuil_min", 5),
        )

```

La classe Mouvement trace chaque entrée ou sortie de stock :

```

# models/mouvement.py
from datetime import datetime

class Mouvement:
    """Enregistrement d'une opération de stock (entrée ou sortie)."""

    TYPES_VALIDES = {"entree", "sortie"}

    def __init__(self, ref_produit: str, type_mvt: str,
                 qte: int, note: str = ""):
        if type_mvt not in self.TYPES_VALIDES:
            raise ValueError(f"type_mvt doit être 'entree' ou 'sortie', reçu : {type_mvt}")
        if qte <= 0:
            raise ValueError(f"qte doit être positive, reçu : {qte}")

        self.ref_produit = ref_produit
        self.type_mvt = type_mvt
        self.qte = qte
        self.note = note
        self.date = datetime.now().isoformat()

    def est_entree(self) -> bool:
        return self.type_mvt == "entree"

    def est_sortie(self) -> bool:
        return self.type_mvt == "sortie"

    def to_dict(self) -> dict:
        return {
            "ref_produit": self.ref_produit,

```

```

        "type_mvt" : self.type_mvt,
        "qte"      : self.qte,
        "note"     : self.note,
        "date"     : self.date,
    }

```

Étape 2 — StockService (20 min)

StockService est le **cerveau** de l'application : il maintient le catalogue en mémoire et persiste les données.

services/stock_service.py

```

import json
from pathlib import Path
from models.produit import Produit
from models.mouvement import Mouvement
from config import DATA_DIR

class StockService:
    """Gère l'ensemble du stock : ajout, entrée, sortie, persistance."""

    def __init__(self):
        self._produits = {} # { ref: Produit } — accès O(1) par ref
        self._mouvements = [] # liste chronologique des mouvements
        self._chemin = DATA_DIR / "stock.json"
        self.charger() # chargement automatique au démarrage

    def ajouter_produit(self, produit: Produit) -> None:
        if produit.ref in self._produits:
            raise ValueError(f"La référence '{produit.ref}' existe déjà.")
        self._produits[produit.ref] = produit
        self.sauvegarder()

    def entree_stock(self, ref: str, qte: int, note: str = "") -> None:
        if qte <= 0:
            raise ValueError("La quantité doit être positive.")
        produit = self.get_produit(ref)
        produit.qte += qte
        self._mouvements.append(Mouvement(ref, "entree", qte, note))
        self.sauvegarder()

    def sortie_stock(self, ref: str, qte: int, note: str = "") -> None:
        if qte <= 0:
            raise ValueError("La quantité doit être positive.")
        produit = self.get_produit(ref)

```

```

    if produit.qte < qte:
        raise ValueError(
            f"Stock insuffisant : {produit.qte} disponible, {qte} demandé."
        )
    produit.qte -= qte
    self._mouvements.append(Mouvement(ref, "sortie", qte, note))
    self.sauvegarder()

def get_produit(self, ref: str) -> Produit:
    if ref not in self._produits:
        raise KeyError(f"Produit '{ref}' introuvable.")
    return self._produits[ref]

def lister_tous(self) -> list:
    return list(self._produits.values())

def produits_en_alerte(self) -> list:
    alertes = []
    for p in self._produits.values():
        if p.est_en_alerte():
            alertes.append(p)
    return alertes

def sauvegarder(self) -> None:
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    data = [p.to_dict() for p in self._produits.values()]
    self._chemin.write_text(
        json.dumps(data, indent=2, ensure_ascii=False),
        encoding="utf-8"
    )

def charger(self) -> None:
    if not self._chemin.exists():
        self._seeder()
        return
    try:
        data = json.loads(self._chemin.read_text(encoding="utf-8"))
        self._produits = {d["ref"]: Produit.from_dict(d) for d in data}
    except (json.JSONDecodeError, KeyError):
        self._seeder()

def _seeder(self) -> None:
    """10 produits de démonstration — certains volontairement en rupture."""
    demo = [
        Produit("CRAY-001", "Crayon HB", "Écriture", 0.15, 0.50, 150, 20),
        Produit("CRAY-002", "Crayon 2B", "Écriture", 0.20, 0.60, 8, 20),
    ]

```

```

        Produit("STYL-001", "Stylo Bleu", "Écriture", 0.30, 0.90, 200, 30),
        Produit("STYL-002", "Stylo Rouge", "Écriture", 0.30, 0.90, 4, 30),
        Produit("GOM-001", "Gomme Blanche", "Effaçage", 0.20, 0.70, 60, 10),
        Produit("PAP-A4", "Rame Papier A4", "Papier", 2.50, 5.00, 300, 50),
        Produit("PAP-A3", "Rame Papier A3", "Papier", 4.00, 8.00, 6, 10),
        Produit("CIS-001", "Ciseaux 17cm", "Coupe", 1.50, 4.00, 25, 5),
        Produit("REG-001", "Règle 30cm", "Mesure", 0.80, 2.00, 40, 10),
        Produit("CAR-001", "Carnet A5", "Papier", 1.20, 3.50, 3, 10),
    ]
    for p in demo:
        self._produits[p.ref] = p
    self.sauvegarder()

```

Étape 3 — Interface graphique (30 min)

L'UI utilise **customtkinter** (version enrichie de Tkinter) et **Treewiew** (ttk) pour le tableau.

```

# ui/app.py (squelette)
import customtkinter as ctk
from services.stock_service import StockService
from ui.frames.stock_frame import StockFrame

class AlQalamApp(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("Al Qalam — Gestion de Stock")
        self.geometry("1100x680")
        self.stock = StockService()
        self._construire_interface()
        self.protocol("WM_DELETE_WINDOW", self._quitter)

    def _construire_interface(self):
        # En-tête bleu
        entete = ctk.CTkFrame(self, fg_color="#1F4E79", height=55, corner_radius=0)
        entete.pack(fill="x")
        ctk.CTkLabel(entete, text="Al Qalam — Gestion de Stock",
                     font=ctk.CTkFont(size=20, weight="bold"),
                     text_color="white").pack(side="left", padx=20, pady=10)

        # Tableau stock
        self.stock_frame = StockFrame(self, self.stock, callbacks={...})
        self.stock_frame.pack(fill="both", expand=True)

# ui/frames/stock_frame.py (Treeview)
from tkinter import ttk

```

```
COLONNES = ("ref", "nom", "categorie", "prix_vente", "qte", "seuil", "statut")
```

```
class StockFrame(ctlk.CTkFrame):
    def __init__(self, parent, stock_service, callbacks):
        super().__init__(parent)
        self._stock = stock_service
        self._callbacks = callbacks
        self._construire_tableau()
        self.rafraichir()

    def _construire_tableau(self):
        self._arbre = ttk.Treeview(self, columns=COLONNES, show="headings")
        for col in COLONNES:
            self._arbre.heading(col, text=col.capitalize())
        self._arbre.pack(fill="both", expand=True)

    def rafraichir(self):
        # Vider et repeupler
        for item in self._arbre.get_children():
            self._arbre.delete(item)
        for p in self._stock.lister_tous():
            statut = "☐ Alerte" if p.est_en_alerte() else "☐ OK"
            self._arbre.insert("", "end", iid=p.ref,
                               values=(p.ref, p.nom, p.categorie,
                                       f"{p.prix_vente:.2f} TND",
                                       p.qte, p.seuil_min, statut))
```

Données de démo — 10 produits

Réf	Nom	Qté	Seuil	Statut
CRAY-001	Crayon HB	150	20	☐ OK
CRAY-002	Crayon 2B	8	20	☐ Alerte
STYL-001	Stylo Bleu	200	30	☐ OK
STYL-002	Stylo Rouge	4	30	☐ Alerte
GOM-001	Gomme Blanche	60	10	☐ OK
PAP-A4	Rame Papier A4	300	50	☐ OK
PAP-A3	Rame Papier A3	6	10	☐ Alerte
CIS-001	Ciseaux 17cm	25	5	☐ OK
REG-001	Règle 30cm	40	10	☐ OK
CAR-001	Carnet A5	3	10	☐ Alerte

Critères de validation

- ☐ `py -3 main.py` lance l'application sans erreur
 - ☐ Le tableau affiche les 10 produits avec leur statut coloré
 - ☐ Bouton "Nouveau" ouvre un dialogue qui ajoute un produit
 - ☐ Bouton "Entrée" augmente la quantité et met à jour le tableau
 - ☐ Bouton "Sortie" réduit la quantité (erreur si stock insuffisant)
 - ☐ Fermer et relancer l'app : les données sont conservées (`data/stock.json`)
 - ☐ Un produit avec `qte < seuil` apparaît en rouge dans le tableau
-

Structure JSON de persistance

```
[
  {
    "ref": "CRAY-001",
    "nom": "Crayon HB",
    "categorie": "Écriture",
    "prix_achat": 0.15,
    "prix_vente": 0.50,
    "qte": 150,
    "seuil_min": 20
  }
]
```

Pour aller plus loin

- Ajouter un champ `description` à `Produit` et l'afficher dans le dialogue de détail
 - Implémenter `supprimer_produit()` avec confirmation
 - Ajouter une barre de statut en bas montrant le nombre total de produits et d'alertes
-

Al Qalam Stock Manager —Formation Python Partie II —Sprint V1